

Codex Course for David

Codex Course Part 1 - Foundations

What Codex is, the prompt formula, plans, goals, threads, and context.

Source note: built from the DOV Codex Course - AI Wisebase Learning Pack. Official Codex source refreshed on 2026-06-29.

Course Promise

By the end, David should be able to:

- Choose the right Codex surface for a task: app, CLI, IDE extension, cloud, Browser, Chrome, Computer Use, or connector.
- Prompt Codex with clear goal, context, constraints, and done-when criteria.
- Use plans, goals, reviews, and tests without overcomplicating the work.
- Understand what skills, plugins, MCP servers, hooks, memories, rules, and automations are for.
- Decide when to let Codex act, when to ask it to inspect first, and when to keep the task read-only.
- Turn repeated workflows into reusable skills or automations.
- Use AI Wisebase as the learning memory: course notes, flashcards, quizzes, knowledge graph, and concept videos.

Module 1 - What Codex Is

Codex is OpenAI's software-development agent. It can write code, understand unfamiliar codebases, review code, debug failures, and automate repetitive development tasks.

Core idea: Codex works best when treated like a configured teammate, not a one-off chatbot.

Practice:

Explain what Codex can do in this workspace. Separate coding, research, file editing, browser testing, app control, and connected-app actions.

Mastery check:

- David can explain the difference between asking ChatGPT a question and asking Codex to work in a local project.
- David can name at least three risks Codex helps manage through sandboxing, approvals, and review.

Sources:

- <https://developers.openai.com/codex/overview>
- <https://developers.openai.com/codex/learn/best-practices>

Module 2 - The Prompt Formula

Use this default structure:

Goal:
Context:
Constraints:
Done when:

Example:

Goal: Create a short guide that teaches me how to use Codex Browser and Computer Use.
Context: I am on Windows in C:\ROOT_OBSIDIAN\DOV and I use Obsidian as my second brain.
Constraints: Keep it beginner-friendly. Separate safe read-only actions from actions that change files or accounts.
Done when: The guide has a 20-minute practice exercise and a checklist I can follow.

Practice:

- Rewrite three vague requests into the four-part format.
- Ask Codex to critique the prompt before doing the task.
- Ask Codex to identify missing context.

Mastery check:

- David can turn a fuzzy idea into a scoped prompt without needing a perfect technical vocabulary.

Sources:

- <https://developers.openai.com/codex/prompting>
- <https://developers.openai.com/codex/workflows>

Module 3 - Plans, Goals, Threads, and Context

Use Plan mode when the work is unclear, risky, or multi-step. Use Goal mode when the task may take many steps and needs a persistent definition of success.

Key terms:

- Thread: one Codex working conversation.
- Local thread: runs on this machine and can use local files and commands.
- Cloud thread: runs in an isolated remote environment.
- Context window: the amount of conversation, files, and tool output Codex can actively hold.
- Compaction: Codex summarizes long context so work can continue.

Practice:

```
/plan
Interview me until this project idea is specific enough to become a Codex goal. Then draft the
final /goal text.
```

Mastery check:

- David can decide whether a task should be a one-off prompt, a plan, a goal, or an automation.

Sources:

- <https://developers.openai.com/codex/prompting>

- <https://developers.openai.com/codex/app/commands>

Checkpoint Quiz 1 - Core Use

1. What are the four parts of the default Codex prompt formula?
2. When should David use Plan mode instead of asking Codex to start editing immediately?
3. What is the difference between a local thread and a cloud thread?
4. Why should "done when" criteria include verification?
5. What is one safe read-only task David can ask Codex to do in this vault?

Answer key:

1. Goal, context, constraints, done when.
2. When the task is unclear, risky, complex, or needs clarification.
3. Local runs on David's machine; cloud runs in an isolated remote environment.
4. So Codex can test or confirm the work instead of only claiming it is done.
5. Inspect relevant notes and report paths without editing files.